

DATENSCHUTZBESTIMMUNGEN

=====

Dokumentstand: Aktiver Proof-of-Concept und Entwicklungsstand.

DevBox besitzt einen funktionsfähigen lokalen Launcher, eine grafische Hauptoberfläche, modulare Strukturansichten, Stammdaten- und Dokumentationspflege, Dokumentations-Snapshots, lokale Werkzeugerkennung, eine Repository-Seite sowie erste Bausteine für den DevBox-spezifischen Veröffentlichungsprozess.

Der deskNode-Bereich ist aktiv in die DevBox integriert. Supervisor und Daemon können aus der GUI gestartet und gestoppt werden; deren Konsolenausgabe wird im deskNode-Log angezeigt. Die Produktversion kann aus den Stammdaten bearbeitet werden. UX-Themes können benannt, angelegt, gelöscht, dupliziert, umbenannt und gespeichert werden. Nach dem Speichern wird die deskNode-Manufakturdatenbank aktualisiert.

Die Symbolverwaltung ist als erste funktionsfähige Katalogoberfläche vorhanden. Sie verwaltet deskNode-Gerätekategorien und Verbrauchergeräte über Auswahlmenüs sowie Dialoge zum Anlegen, Bearbeiten und Löschen. Neue Geräte erhalten eine einzelne PNG-Quelle, die unter ihrer record_id als symbol_source_<record_id>.png abgelegt wird. Nach jeder erfolgreichen Katalogänderung wird create_manufacturer_db.py ausgeführt, damit mnfctr_db.r0b die aktuellen Theme-, Kategorien- und Gerätedaten enthält.

Der Graphic-Pack-Build ist als Proof-of-Concept funktionsfähig. Er erzeugt aus Symbolquellen und UX-Themes mehrere vorbereitete Grafikzustände für Verbraucher. Die eigentliche deskNode-Laufzeitintegration, Asset-Auswahl, Sprachpakete und vollständige Zustandslogik werden noch weiterentwickelt.

DevBox ist keine fertige Endnutzer- oder Release-Version. Oberflächen, Datenbankmigrationen, Produktmodule, Veröffentlichungsabläufe, Dokumentvorlagen und Tests werden fortlaufend überarbeitet. Diese Dokumentation beschreibt den derzeitigen Stand und muss bei funktionalen Änderungen mitgepflegt werden.

Erstveröffentlichung: Verantwortliche Stelle / Herausgeber: Markus Walloner Name: Markus Walloner Land: Germany (DE)

1. Gegenstand

Diese Datenschutzbestimmungen beziehen sich auf die in der begleitenden Projektdokumentation beschriebene Software bzw. das beschriebene Projekt.

Kurzbeschreibung: DevBox ist die lokale Entwicklungszentrale der CYXnTrol Development Platform. Die PySide6-Anwendung bündelt Projektstruktur, Stammdaten, Dokumentation, lokale Werkzeuge, Produktmodule, Repository-Vorbereitung und wiederkehrende Entwicklungsabläufe.

DevBox ist kein Endnutzerprodukt. Es ist eine interne Arbeitsumgebung, in der technische Anforderungen, Datenmodelle, Oberflächen, Build-Schritte und Veröffentlichungsstände nachvollziehbar vorbereitet, geprüft und weiterentwickelt werden.

2. Datenschutzbestimmungen

DATENSCHUTZHINWEIS – ENTWURFSFASSUNG

1. Zweck und Geltungsbereich

Dieser Datenschutzhinweis beschreibt den derzeit vorgesehenen lokalen Betrieb von DevBox. DevBox ist eine lokale Entwicklungsumgebung. Nach aktuellem Konzept nutzt sie ohne ausdrückliche Nutzeraktion keine Telemetrie, Werbung, Analyse-Dienste oder proprietären Cloud-Dienste.

2. Verantwortliche Stelle und Kontakt

Verantwortliche Stelle für den DevBox-Entwicklungsstand ist Markus Walloner im Rahmen von CYX Labs. Rückfragen zu diesem Projekt können über den im jeweiligen Repository oder in den begleitenden Projektunterlagen genannten Kontaktweg gestellt werden.

3. Lokal verarbeitete Daten

DevBox verarbeitet insbesondere lokal:

- Pfade zum Projektroot, zu Projektdateien und zu lokalen Werkzeugen;
- Inhalte lokaler SQLite-Datenbanken, einschließlich Produkt-, Hersteller-, Dokumentations-, Repository-, Struktur-, UX-Theme-, Kategorie- und Verbraucher-Metadaten;
- lokale PNG-Quellen für Verbrauchergeräte, abgeleitete deskNode-Manufakturdatenbanken und vorbereitete Grafikpakete;
- Runtime- und Fehlerprotokolle in logfile.r0b;
- erkannte Pfade zu Inkscape und GIMP in locdata.r0b;
- vom Nutzer eingegebene Versionsnummern, Theme-Namen, RGBA-Werte, Kategorienamen, Gerätekennungen und weitere Oberflächeneinstellungen;
- Commit-Texte sowie Bilddateipfade und Bilddateien für ausdrücklich ausgelöste Repository-Vorgänge;
- technische Ausgabe lokaler Unterprozesse, zum Beispiel von Python, Git, Inkscape, GIMP oder produktbezogenen Skripten.

Diese Daten werden grundsätzlich lokal in Projektordnern, temporären Arbeitsbereichen oder unter AppData verarbeitet und dienen der vom Nutzer ausgelösten DevBox-Funktion.

4. Temporäre Arbeitsbereiche

Für Dokumentexporte, Grafik-Builds, Repository-Vorbereitung und ähnliche Vorgänge erzeugt DevBox eindeutige temporäre Arbeitsordner. Dort können Kopien von Projektdateien, Zwischengrafiken, Masken, generierte Dokumente und vorbereitete Assets verarbeitet werden. Die Anwendung versucht, diese Arbeitsordner nach Abschluss zu entfernen. Bei fehlgeschlagenem Cleanup können einzelne temporäre Dateien bis zu einer späteren manuellen oder systemseitigen Bereinigung verbleiben.

5. Externe Programme

DevBox kann nach expliziter Nutzeraktion lokale Programme wie Inkscape, GIMP, Git, Git Credential Manager oder Installer starten. Ab dem Start gelten zusätzlich die eigenen Datenverarbeitungen und Nutzungsbedingungen dieser Programme. DevBox speichert keine Git-Passwörter oder Zugriffstokens als eigene Repository-Zugangsdaten.

6. Repository-Vorgänge und Datenübertragung

Wenn ein Push-to-Git-Vorgang ausdrücklich ausgelöst wird, überträgt Git den vom Nutzer ausgewählten Repository-Inhalt, Commit-Informationen und technisch erforderliche Verbindungsdaten an den jeweiligen Repository-Anbieter. Empfänger, Repository-Sichtbarkeit und Branch ergeben sich aus der vom Nutzer gepflegten Repository-Konfiguration. Der Nutzer ist dafür verantwortlich, keine personenbezogenen oder vertraulichen Daten unbeabsichtigt zu veröffentlichen.

7. Rechtsgrundlage und Prüfung

Dieser Text ist ein technischer Entwurf und keine individuelle Rechtsberatung. Vor einer öffentlichen oder kommerziellen Verteilung muss er auf reale Datenflüsse, Kontaktangaben, mögliche Rechtsgrundlagen, Auftragsverarbeitung, Drittlandtransfers und anwendbares Datenschutzrecht geprüft und angepasst werden.

3. Projektkontext

Zweck: DevBox soll wiederkehrende und fehleranfällige Entwicklungsarbeit in nachvollziehbare lokale Werkzeuge überführen. Dazu gehören insbesondere die Pflege von Produkt- und Herstellerdaten, strukturierte Dokumentation, vorbereitete Veröffentlichungsschritte, die Integration lokaler Kreativprogramme sowie die kontrollierte Ausführung produktbezogener Entwicklungsfunktionen.

Die Anwendung schafft eine gemeinsame Arbeitsgrundlage für Produkte der CYXnTrol Development Platform. Statt Abläufe nur als Erinnerung, Ordnerkonvention oder Sammlung einzelner Konsolenbefehle zu halten, werden sie als Daten, Skripte, GUI-Funktionen und überprüfbare Prozessketten festgehalten.

Für deskNode dient DevBox zusätzlich als Entwicklungs- und Konfigurationsoberfläche für Produktversionen, UX-Themes, Gerätekategorien, Verbrauchersymbole und vorbereitete Grafikpakete. Die spätere deskNode-Laufzeit soll daraus reproduzierbare Daten und bereits gerenderte Assets erhalten.

Konfiguration: Die Konfiguration ist in mehrere Ebenen getrennt.

Projektroot: Der Projektroot wird über die .root-Datei gefunden. Er ist die maßgebliche Quelle für Skripte, Ressourcen, Formarchive, Installer, globale Fonts und die zentrale DevBox-Datenbank.

Zentrale Stammdaten: resources/organization/devbox_db.r0b enthält Hersteller-, Produkt-, Dokumentations-, Struktur-, Repository-, UX-Theme- und Symbolkataloginformationen. Die Tabelle "ux-deskNode" enthält mehrere benannte Theme-Datensätze mit record_id und theme_name. Die Tabellen

desknode_consumer_device_categories und desknode_consumer_devices enthalten die dauerhaften technischen Kategorien und Verbrauchergeräte. Schemaerweiterungen sollen vorhandene Datensätze erhalten und fehlende Felder über Migration ergänzen.

Symbolkatalog: Kategorien werden über category_key und einen automatisch abgeleiteten translation_key geführt. Geräte werden über device_key, category_id und einen automatisch abgeleiteten translation_key geführt. Beim Anlegen eines Geräts wird eine einzelne PNG gewählt oder abgelegt und nach resources/graphics/symbol_source_<record_id>.png kopiert. Die Zuordnung zur Kategorie erfolgt über category_id, nicht über einen Dateinamen.

deskNode-Laufzeitdaten: applications/deskNode/data/mnfctr_db.r0b wird durch create_manufacturer_db.py neu erzeugt. Sie enthält master_data, ux_themes, consumer_device_categories und consumer_devices. Nach jeder erfolgreichen Änderung an Version, Theme, Kategorie oder Verbrauchergerät wird diese Aktualisierung ausgeführt.

Oberflächengestaltung: Farben werden als achtstellige RGBA-Hexwerte ohne # gespeichert, zum Beispiel 00e4ffff. Schriftdateien werden rekursiv aus resources/fonts gelesen und als projektroot-relative Pfade gespeichert. Schriftrollen umfassen große Überschriften, Bereichsüberschriften, Standardtext, Schaltflächentext, Eingabefeldtext, Statusmeldungen und Protokolltext. Weitere Theme-Werte betreffen Schriftgrößen, Schriftschnitt, Unterstreichungen, Konturen, Radien und Zustandsfarben.

Lokale Runtime-Daten: %appdata%/CYX Labs/CYXnTrol/DevBox/logfile.r0b enthält Logdaten. %appdata%/CYX Labs/CYXnTrol/DevBox/locdata.r0b enthält geprüfte Pfade für Inkscape und GIMP.

Werkzeugerkennung: Gespeicherte Pfade werden beim Start geprüft. Ungültige Pfade werden erneut gesucht, zunächst unter Program Files und anschließend im System-Temp-Bereich. Fehlende Werkzeuge beschränken nur die jeweiligen Funktionen.

Repository-Daten: Repository-URL und Branch werden pro Produkt in product_credentials gepflegt. Git-Zugangsdaten werden nicht in DevBox gespeichert. Für den DevBox-Push wird ein temporärer Veröffentlichungsroot erzeugt; der Produktordner applications wird darin bereinigt und anschließend gezielt mit applications/deskNode/resources/scripts ohne __pycache__-Inhalte befüllt.

Technologie: DevBox verwendet derzeit vor allem folgende Technologien:

- Python als Kernsprache für Launcher, Prozesslogik, Datenaufbereitung, Automationen und produktbezogene Werkzeuge.
- PySide6 für die lokale grafische Benutzeroberfläche.
- SQLite für zentrale Projektstammdaten, deskNode-Manufakturdaten sowie lokale Runtime-, Log- und Standortdaten.
- openpyxl als Übergangs- oder Importwerkzeug für Tabellen, nicht als zentrale Stammdatenquelle.
- ReportLab und svglib für die lokale Erzeugung gestalteter PDF-Dokumente.
- Git und Git Credential Manager für Repository-Vorgänge und Authentifizierung.
- Inkscape für Skalierung, SVG-Arbeit und Vektorisierung von Masken.

- GIMP 3 für nichtinteraktive Bildbearbeitung und vorbereitende Maskenschritte.
- XML-/SVG-Verarbeitung mit Python-Standardbibliotheken für die Bereinigung generierter Masken.
- C#-Generierung, .NET SDK, WiX und perspektivisch weitere Packaging-Werkzeuge für Build- und Installer-Prozesse.
- temporäre Arbeitsbereiche unter dem System-Temp-Verzeichnis für kontrollierte Kopien, Grafik-Builds, Exporte und Veröffentlichungsvorbereitung.

Die technische Struktur trennt GUI, Funktionsskripte, Subscripts, Datenquellen, Produktdatenbanken, externe Werkzeuge und temporäre Arbeitsstände möglichst klar. Der deskNode-Symbolkatalog verknüpft SQLite-Datensätze mit PNG-Quellen über stabile record_id-Werte, während der Graphic-Pack-Build die daraus resultierenden visuellen Varianten erzeugt.

Repository-Hinweis: Das DevBox-Repository repräsentiert die CYXnTrol Development Plattform selbst, nicht ein fertiges Endnutzerprodukt. Es dient als Entwicklungsstand, transparente Arbeitsprobe und Grundlage für die Pflege der Plattform.

Der veröffentlichte Stand wird nicht direkt aus dem produktiven Projektroot kopiert. Für DevBox wird ein gesonderter bereinigter root_dir-Stand erzeugt. Er enthält vorgesehenen Quellcode, Dokumente und Assets, während lokale Laufzeitdaten, temporäre Reste, Installer, unnötige Build-Ausgaben und nicht veröffentlichte Daten außerhalb des Veröffentlichungspakets bleiben.

Der temporäre Veröffentlichungsroot behandelt applications als bewusstes Auslieferungsfenster: Der Bereich wird zunächst bereinigt. Danach wird applications/deskNode/resources/scripts einschließlich enthaltener Dateien in den Veröffentlichungsstand kopiert. __pycache__-Ordner sowie .pyc- und .pyo-Dateien bleiben ausgeschlossen. Der echte lokale Produktordner wird dadurch nicht verändert.

DevBox wird in einem KI-gestützten, iterativen Workflow entwickelt. Anforderungen, Architektur, Datenmodelle, UX-Entscheidungen, Testfälle und Abnahmen werden durch den Projektverantwortlichen gesteuert. Die Implementierung entsteht mit KI-gestützten Entwicklungswerkzeugen und wird gegen die definierten Funktionsanforderungen geprüft.

Copyright (c) 2026 Markus Walloner